

Project Report For Secure Complaint/Whistleblower System

**Ensuring Confidentiality and Data Integrity Through AES Encryption
and Blockchain in a Complaint System**

Submitted by

Muhammad Rizwan Shafiq, SP24-BCS-069

Rehmooz Shahbaz, SP24-BCS-105

Department of Computer Science, CU, Islamabad.

20th October, 2025

Table of Contents

1. Introduction	1
2. System Overview	1
Key Principles Used:.....	1
3. Technologies Used	1
4. System Workflow	1
5. Core Functions with Code & Explanation.....	2
5.1 Hashing & Helpers (Passwords, Text, Masking)	2
5.2 AES Key, Padding, Encrypt/Decrypt.....	2
5.3 Block Structure & Proof-of-Work Mining	3
5.4 Blockchain Operations.....	4
5.5 Application Layer (Users, Admin, Submissions, Export)	5
6. AES Encryption Explained (Simple)	7
7. Blockchain & Proof of Work	7
Mining (Proof of Work)	7
8. Security Features	8
9. Conclusion	8
10. References.....	8

1. Introduction

This project is a Secure Complaint / Whistleblower System designed using Python. It ensures that users can submit confidential complaints securely. To protect privacy, the system uses AES encryption for confidentiality and blockchain technology for data integrity and immutability. Each complaint is securely stored and cannot be altered once recorded.

The system addresses real-world needs where students or employees may want to report issues anonymously while ensuring their identity and message remain protected.

2. System Overview

The system allows: - User registration and login (authentication) - Secure complaint submission (encrypted using AES) - Storage of complaints in a blockchain for immutability - Admin review of decrypted complaints with authentication

Key Principles Used:

- **Confidentiality:** AES-CBC Encryption
- **Integrity:** SHA-256 Hashing
- **Authentication:** Password Hashing & User Login
- **Immutability:** Blockchain with Proof of Work

3. Technologies Used

Technology	Purpose
Python	Overall implementation
AES (PyCryptodome)	Encrypt complaints
SHA-256 (hashlib)	Secure password hashing and data integrity
Blockchain (Custom)	Store encrypted complaints securely
Proof of Work	Prevent tampering with stored complaints

4. System Workflow

1. **User Registration:** User creates an account with an ID and password.
2. **Authentication:** Login required before complaint submission.
3. **AES Encryption:** Complaint text is encrypted using AES-CBC.
4. **Blockchain Storage:** Complaint is stored as a block with hash links.
5. **Admin Review:** Admin verifies password to decrypt and view complaints.

5. Core Functions with Code & Explanation

5.1 Hashing & Helpers (Passwords, Text, Masking)

Function: *sha256_hex()*

```
def sha256_hex(b: bytes) -> str:
    return hashlib.sha256(b).hexdigest()
```

Why: Standard SHA-256 hash for bytes → hex string. Used everywhere (passwords, key hash, data integrity, block hash).

Function: *hash_text()* & *hash_password()*

```
def hash_text(s: str) -> str:
    return sha256_hex(s.encode())

def hash_password(pw: str) -> str:
    return sha256_hex(pw.encode())
```

Why: Convenience wrappers for strings. We **never** store plain passwords, only the SHA-256 hash.

Function: *mask_user_id()*

```
def mask_user_id(uid: str) -> str:
    return uid[:2] + "*"*(max(0, len(uid) - 4)) + uid[-2:] if len(uid) > 4 else "*****"
```

Why: Protects user identity in CLI/blockchain view by partially hiding the ID.

5.2 AES Key, Padding, Encrypt/Decrypt

Function: *kdf_key()* (Key Derivation)

```
def kdf_key(master_key_str: str) -> bytes:
    return hashlib.sha256(master_key_str.encode()).digest()
```

Why: AES needs a fixed-length binary key (here **32 bytes** for AES-256). We convert a readable string into a strong key using SHA-256.

Functions: *pkcs7_pad()* / *pkcs7_unpad()*

```
def pkcs7_pad(data: bytes, block_size: int = 16) -> bytes:
    pad_len = block_size - (len(data) % block_size)
    if pad_len == 0:
        pad_len = block_size
    return data + bytes([pad_len]) * pad_len

def pkcs7_unpad(data: bytes, block_size: int = 16) -> bytes:
    if not data or len(data) % block_size != 0:
        raise ValueError("Invalid padded data length")
    pad_len = data[-1]
    if pad_len < 1 or pad_len > block_size:
```

```

        raise ValueError("Invalid padding")
    if data[-pad_len:] != bytes([pad_len]) * pad_len:
        raise ValueError("Bad padding bytes")
    return data[:-pad_len]

```

Why: AES-CBC works on 16-byte blocks. PKCS#7 adds/removes padding so encryption works for any message length.

Functions: aes_encrypt_pt() / aes_decrypt_ct()

```

def aes_encrypt_pt(plaintext: str, key_bytes: bytes) -> Tuple[bytes, bytes]:
    iv = get_random_bytes(16)
    cipher = AES.new(key_bytes, AES.MODE_CBC, iv)
    ct = cipher.encrypt(pkcs7_pad(plaintext.encode(), 16))
    return iv, ct

```

```

def aes_decrypt_ct(iv: bytes, ciphertext: bytes, key_bytes: bytes) -> str:
    cipher = AES.new(key_bytes, AES.MODE_CBC, iv)
    pt = pkcs7_unpad(cipher.decrypt(ciphertext), 16)
    return pt.decode()

```

Why: - Encrypt: generate a fresh random **IV**, pad, encrypt.

- Decrypt: use the stored IV + ciphertext to get plaintext back, then unpad.

5.3 Block Structure & Proof-of-Work Mining

Dataclass: Block

```

@dataclass
class Block:
    index: int
    timestamp: float
    user_mask: str
    uid_hash: str          # sha256(uid)
    key_hash: str          # sha256(master_key_str)
    iv_hex: str            # AES IV (hex)
    enc_hex: str           # ciphertext (hex)
    data_hash: str         # sha256(ciphertext bytes)
    prev_hash: str
    nonce: int = 0
    hash: str = ""

```

Why: A block holds the masked identity, encrypted complaint, integrity hash (data_hash), a link to the previous block, and a **nonce** for mining.

Method: compute_hash()

```

def compute_hash(self) -> str:
    s = (
        f"{self.index}{self.timestamp}{self.user_mask}{self.uid_hash}"

```

```

        f"{self.key_hash}{self.iv_hex}{self.enc_hex}{self.data_hash}"
        f"{self.prev_hash}{self.nonce}"
    )
    return sha256_hex(s.encode())

```

Why: Computes the block's fingerprint. Any change in block fields → a different hash → tamper detection.

Method: mine() (Proof of Work)

```

def mine(self, difficulty: int):
    prefix = "0" * difficulty
    while True:
        self.hash = self.compute_hash()
        if self.hash.startswith(prefix):
            return
        self.nonce += 1

```

Why: Tries different nonce values until the hash starts with difficulty zeros (e.g., "00"). This makes altering history computationally expensive.

5.4 Blockchain Operations

Constructor & Genesis

```

class Blockchain:
    def __init__(self, difficulty: int = 2):
        self.difficulty = difficulty
        g = Block(
            index=0,
            timestamp=time.time(),
            user_mask="GENESIS",
            uid_hash="",
            key_hash="",
            iv_hex="",
            enc_hex="",
            data_hash="",
            prev_hash="0" * 64
        )
        g.hash = g.compute_hash()
        self.chain = [g]

```

Why: Creates the **genesis block** (index 0). It anchors the chain.

Method: add_block()

```

def add_block(self, **fields):
    last = self.chain[-1]
    b = Block(index=len(self.chain), timestamp=time.time(),
        prev_hash=last.hash, **fields)
    b.mine(self.difficulty)
    self.chain.append(b)
    return b

```

Why: Links a new block to last.hash, mines it, then appends to the chain.

Method: is_valid()

```
def is_valid(self) -> bool:
    for i in range(1, len(self.chain)):
        cur = self.chain[i]
        prev = self.chain[i - 1]
        if cur.prev_hash != prev.hash:
            return False
        if cur.compute_hash() != cur.hash:
            return False
    return True
```

Why: Recomputes and checks every link. If anything was edited, the chain fails validation.

5.5 Application Layer (Users, Admin, Submissions, Export)

Register User — register_user()

```
def register_user(self, uid: str, password: str):
    if uid in self.users:
        raise ValueError("User already exists.")
    self.users[uid] = {
        'pw_hash': hash_password(password),
        'has_submitted': False,
    }
```

Why: Saves a new user with a **hashed** password and a flag ensuring only **one complaint per user**.

Authenticate User — auth_user()

```
def auth_user(self, uid: str, password: str) -> bool:
    user_record = self.users.get(uid)
    if not user_record:
        return False
    return user_record['pw_hash'] == hash_password(password)
```

Why: Compares the hash of the entered password with the stored hash.

Change Admin Password — set_admin_password()

```
def set_admin_password(self, old_pw: str, new_pw: str):
    if self.admin_pw_hash != hash_password(old_pw):
        raise PermissionError("Admin authentication failed (wrong current password).")
    self.admin_pw_hash = hash_password(new_pw)
```

Why: Secure update. Admin must prove they know the **current** password.

Submit Complaint — submit_complaint() (AES path)

```
def submit_complaint(self, uid: str, password: str, text: str):
    if uid not in self.users:
```

```

        raise PermissionError("User not registered. Please register
first.")
    if not self.auth_user(uid, password):
        raise PermissionError("Authentication failed.")
    if self.users[uid]['has_submitted']:
        raise PermissionError("Already submitted a complaint.")

    iv, ct = aes_encrypt_pt(text, self.master_key_bytes)
    data_hash = sha256_hex(ct)

    self.bc.add_block(
        user_mask=mask_user_id(uid),
        uid_hash=hash_text(uid),
        key_hash=self.master_key_hash,
        iv_hex=iv.hex(),
        enc_hex=ct.hex(),
        data_hash=data_hash
    )
    self.users[uid]['has_submitted'] = True

```

Why: Enforces auth **before** taking input, encrypts with AES, stores iv_hex, enc_hex, and data_hash on-chain, and mines the new block.

Admin Review — admin_list_decrypted()

```

def admin_list_decrypted(self, password: str):
    if not self.auth_admin(password):
        raise PermissionError("Admin authentication failed.")
    if not self.bc.is_valid():
        raise RuntimeError("Blockchain invalid.")

    out = []
    for b in self.bc.chain[1:]: # skip genesis
        status = "ok" if sha256_hex(bytes.fromhex(b.enc_hex)) ==
b.data_hash else "tampered"
        text = "<undecryptable>"
        if status == "ok" and b.key_hash == self.master_key_hash:
            try:
                iv = bytes.fromhex(b.iv_hex)
                ct = bytes.fromhex(b.enc_hex)
                text = aes_decrypt_ct(iv, ct, self.master_key_bytes)
            except Exception:
                text = "<decrypt-error>"
        out.append({
            "user_mask": b.user_mask,
            "status": status,
            "text": text,
            "block": b.hash[:10] + "..."
        })
    return out

```


Why: Only an authenticated admin can view. It first checks chain validity and ciphertext integrity, then decrypts using the **same key** and stored **IV**.

Show Chain — show_blockchain()

```
def show_blockchain(self):
    for b in self.bc.chain[:]:
        print(f"Idx:{b.index} Prev:{b.prev_hash[:8]} Hash:{b.hash[:8]}
        User:{b.user_mask} H:{b.data_hash[:8]}")
```

Why: Displays a compact view of each block for quick inspection.

Export Chain — export_chain_json() + CLI save

```
def export_chain_json(self) -> str:
    return json.dumps([asdict(b) for b in self.bc.chain], indent=2)
```

Why: Serializes the entire chain to JSON (printed on CLI **and** saved to file by the menu option).

List Users — list_users()

```
def list_users(self):
    return [{"uid": uid, "mask": mask_user_id(uid), "submitted":
    rec['has_submitted']} for uid, rec in self.users.items()]
```

Why: Shows masked IDs and whether each user has submitted already.

6. AES Encryption Explained (Simple)

AES (Advanced Encryption Standard) is a modern encryption method used worldwide. In this project, we use **AES-CBC mode** with: - **Master Key:** Defined in code - **IV (Initialization Vector):** Random for each complaint - **Padding:** PKCS#7 to align data to 16 bytes. We store **iv_hex + enc_hex + data_hash** on-chain so we can verify and decrypt correctly later.

Only someone with the same key can decrypt and read the complaint.

7. Blockchain & Proof of Work

Each complaint becomes a **block** in the blockchain. The block contains: - **enc_hex** – Encrypted complaint - **iv_hex** – IV used for AES - **data_hash** – SHA-256 hash of encrypted data - **prev_hash** – Hash of previous block

Mining (Proof of Work)

Before a block is added, it must find a hash starting with 00. This prevents tampering.

- Every complaint is a **block** with **prev_hash** pointing to the previous block.
- **Mining** tries different nonce values until the block hash starts with "0" * difficulty (e.g., "00").

- If anyone modifies `enc_hex` or `iv_hex`, then `data_hash` and the block hash won't match; **validation fails**.

8. Security Features

Goal	How We Achieve It
Authentication	Store only SHA-256 password hashes; verify on login
Confidentiality	AES-CBC with PKCS#7 padding and random IV
Integrity	Store SHA-256 of ciphertext (detects tampering)
Immutability	Blockchain with PoW and hash links
Privacy	Masked IDs + store only <code>uid_hash</code> on-chain

9. Conclusion

This project demonstrates how cryptography and blockchain can be combined to build a secure digital complaint/whistleblower system. It ensures that messages remain private, untampered, and traceable only by admin.

10. References

- **PyCryptodome AES Official Documentation**
<https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html>
- **SHA-256 Hashing – Python `hashlib` Documentation**
<https://docs.python.org/3/library/hashlib.html>
- **Blockchain Basics (Proof of Work Concept)**
<https://www.investopedia.com/terms/p/proof-work.asp>
- **AES Encryption & Decryption Guide in Python**
<https://onboardbase.com/blog/aes-encryption-decryption/>
- **Beginner Guide: Encryption & Decryption in Python**
<https://medium.com/@sunilnepali844/complete-guide-to-encryption-and-decryption-in-python-for-beginners-61c2343c3f2b>
- **Python File Handling (Recommended Practical Video)**
<https://www.youtube.com/watch?v=Uh2ebFW8OYM>